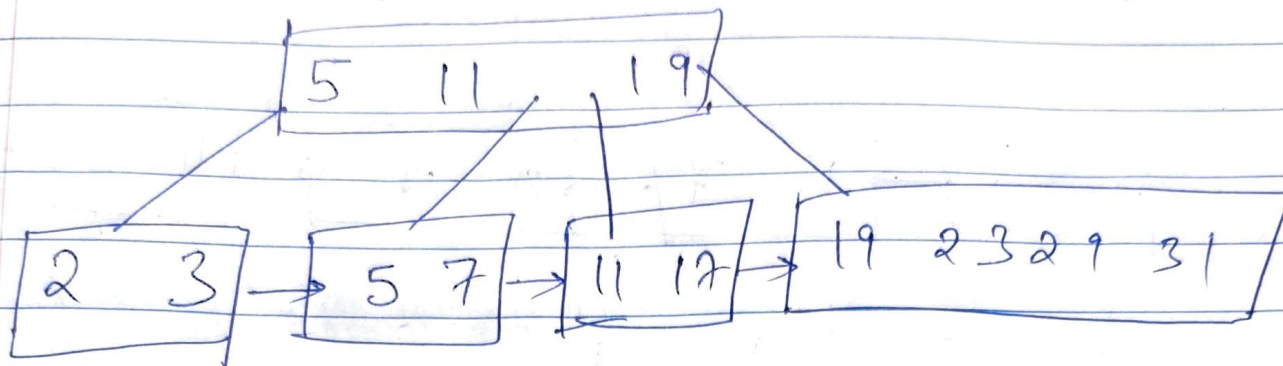


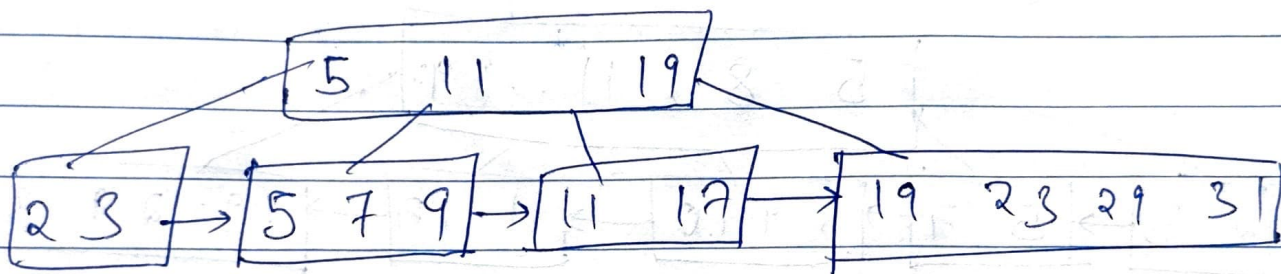
Assignment 3

Index and Query Processing

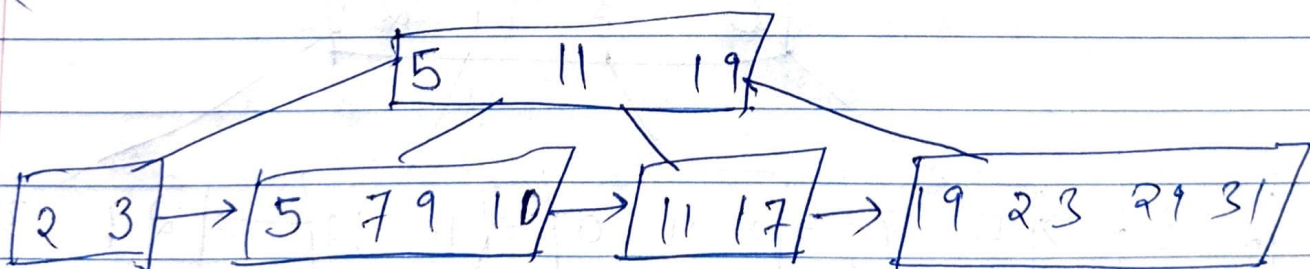
(1) (2, 3, 5, 7, 11, 17, 19, 23, 29, 31)



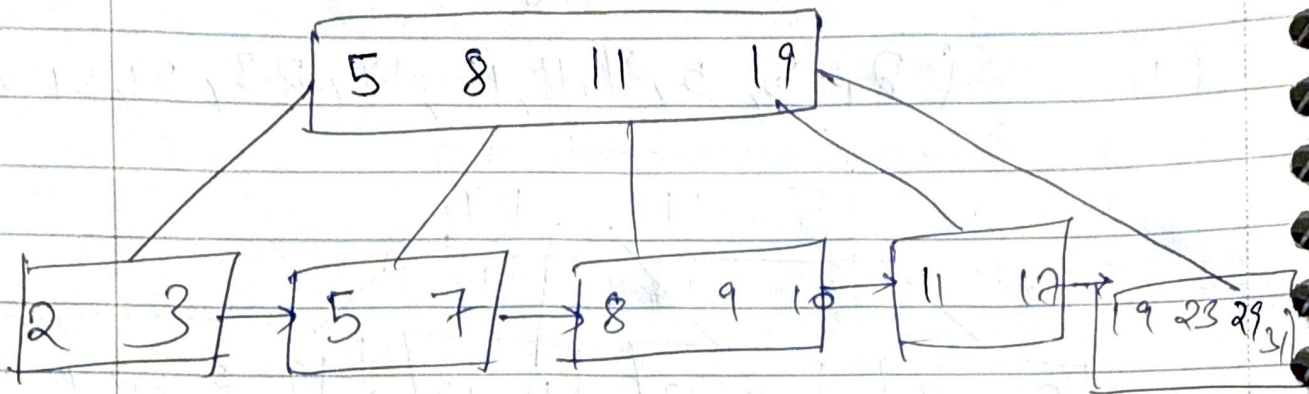
(2) (a) Insert 9



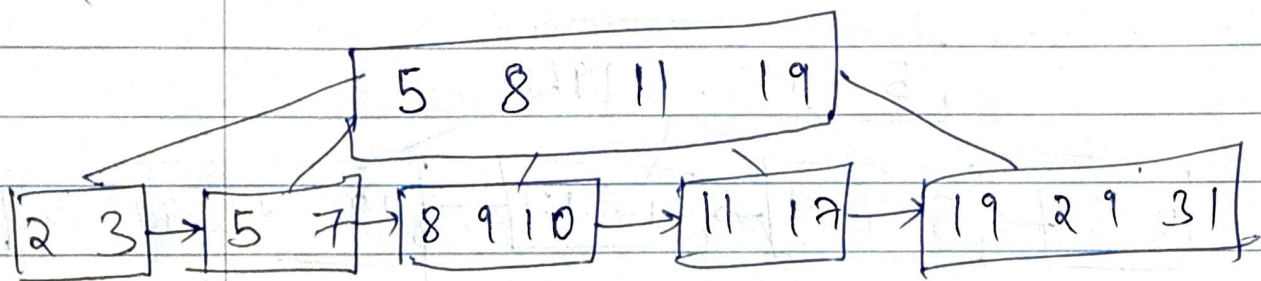
(b) Insert 10



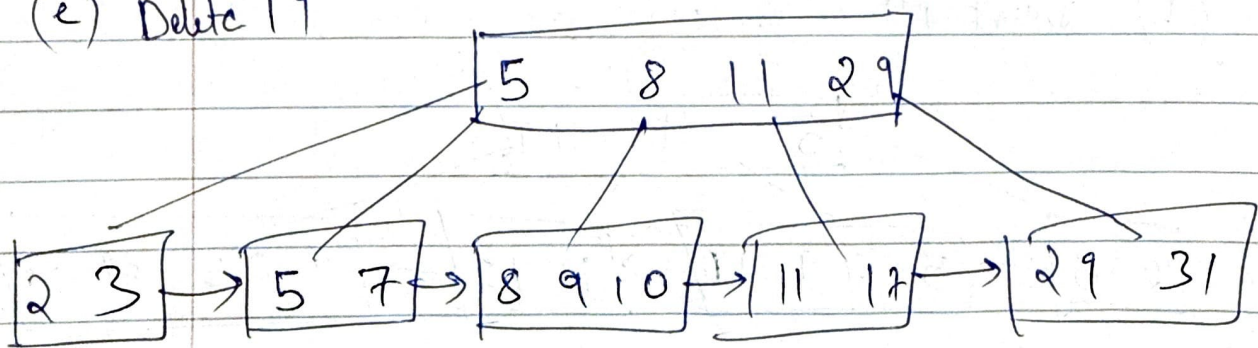
(c) Insert 8



(d) Delete 23



(e) Delete 19



(3ans) Pass ① (initial runs):

- (1) (emu, 1), (kangaroo, 17), (wallaby, 21)
- (2) (lion, 8), (platypus, 3), (wombat, 13)
- (3) (meerkat, 6), (warthog, 4), (zebra, 11)
- (4) (baboon, 12), (hornbill, 2), (hyena, 9)

Pass 1 (after merging pairs):

- (1) (emu, 1), (kangaroo, 17), (lion, 8), (platypus, 3),
(wallaby, 21), (wombat, 13)
- (2) (baboon, 12), (hornbill, 2), (hyena, 9), (meerkat, 6),
(warthog, 4), (zebra, 11)

Pass 2 (final merge):

- (1) (baboon, 12), (emu, 1), (hornbill, 2), (hyena, 9),
(kangaroo, 17), (lion, 8), (meerkat, 6), (platypus, 3),
(wallaby, 21), (warthog, 4), (wombat, 13), (zebra, 11)

(4 ans)

$\pi_{T.branch_name} ((\pi_{T.branch_name, assets} (P_T(branch)))$

$\bowtie_{T.assets \supseteq S.assets}$

$(\pi_{assets} (\sigma_{C.branch_city = 'Brooklyn'} (P_S(branch))))$

This expressions performs the theta join on the smallest amount of data possible. It does this by restricting the right-handed side operand of the join to only those branches in Brooklyn and also eliminating the unneeded attributes from both the operands.

(5 ans)

r_1 needs 800 blocks, and r_2 needs 1500 blocks. Let us assume M pages of memory. If $M > 800$, the join can easily be done in $1500 + 800$ disk accesses, using even plain nested-loop join. So we consider only the case where $M \leq 800$ pages.

(a) Nested-loop join:

Using r_1 as the outer relation, we need $2000 * (1500 + 800) = 30,000,800$ disk accesses. If r_2 is the outer relation, we need $4500 * 800 + 1500 = 36,001,500$ disk accesses.

(b) Block nested-loop join:

If r_1 is the outer relation, we need $\left\lceil \frac{800}{M-1} \right\rceil \times \frac{1500 + 800}{800}$ disk accesses. If r_2 is the outer relation, we need

$$\left\lceil \frac{1500}{M-1} \right\rceil \times 800 + 1500 \text{ disk accesses.}$$

(c) Merge join:

Assuming that r_1 and r_2 are not initially sorted on the join key, the total sorting cost inclusive of the output is $B_S = 1500 (2 \lceil \log_{M-1} (1500/M) \rceil + 2) +$

$800 (2 \lceil \log_{M-1} (800/M) \rceil + 2)$ disk accesses. Assuming all tuples with the same value for the join attributes fit in memory, the total cost is $B_S + 1500 + 800$ disk accesses.

(d) Hash join:

We assume no overflow occurs. Since r_1 is smaller, we use it as the build relation and r_2 as the probe relation. If $M > 800/M$, i.e., no need for recursive partitioning, then the cost is $3(1500 + 800) = 6900$ disk accesses, else the cost is $2(1500 + 800) \lceil \log_{M-1} (800) - 1 \rceil + 1500 + 800$ disk accesses.

(6 ans)

If there are multiple tuples in the inner relation with the same value for the join attributes, we may have to access that many blocks of the inner relation for each tuple of the outer relation. That is why it is inefficient. To reduce this cost we can perform a join of the outer relation tuples with just the secondary index leaf entries, postponing the inner relation tuple retrieval. The result file obtained is then sorted on the inner relation addresses, allowing an efficient physical order scan to complete the join.

Hybrid merge-join requires the outer relation to be sorted. The above algorithm does not have this requirement, but for each tuple in the outer relation it needs to perform an index lookup on the inner relation. If the outer relation is much larger than the inner relation, this index lookup cost will be less than the sorting cost, thus this algorithm will be more efficient.

(7 ans)

We can store the entire smaller relation in memory, read the larger relation block by block, and perform nested-loop join using the larger one as the outer relation. The number of I/O operations is equal to $b_r \times b_s$, and the memory requirement is $\min(b_r, b_s) + 1$ pages.

(8am) (a) Use the index to locate the first tuple whose branch-city field has value "Brooklyn". From this tuple, follow the pointer chains till the end, visiting all the tuples.

(b) For this query, the index serves no purpose. We can scan the file sequentially and select all tuples whose branch-city field is anything other than "Brooklyn".

(c) This query is equivalent to the query

$\sigma(\text{branch-city} \geq \text{'Brooklyn'} \wedge \text{assets} < 5000)(\text{branch})$

Using the branch-city index, we can retrieve all tuples with branch-city value greater than or equal to "Brooklyn" by following the pointer chains from the first "Brooklyn" tuple. We also apply the additional criteria of $\text{assets} < 5000$ on every tuple.

(9 ans)

Let outer be the iterator which returns successive tuples from the pipelined outer relation. Let inner be the iterator which returns successive tuples of the inner relation having a given value at the join attributes. The inner iterator returns these tuples by performing an index lookup. The functions IndexedNLJoin::Open, IndexedNLJoin::Close and IndexedNLJoin::next to implement the indexed nested loop join iterator are given below. The 2 iterators outer and inner, the value of the last read outer relation tuple t_r and a flag $done_r$ indicating whether the end of the outer relation scan has been reached are the state information which need to be remembered by IndexedNLJoin between calls.

(10 ans)

Producer-driven pipelining executes the same set of instructions to generate multiple tuples by consuming already generated tuples from the inputs. Thus instruction cache hits will be more. In comparison, demand-driven pipelining switches from the instructions of one function to another for each tuple, resulting in more misses.

By generating multiple results at one go, a $\text{rad}()$

function would receive multiple tuples as its inputs and have a loop that generates multiple tuples for its outputs without switching execution to another function. Thus, the instruction cache hit rate can be expected to improve.

(Hans) Let S be a set of n keywords. An algorithm to find all documents that contain at least k of these keywords is given in

```
initialize the list  $L$  to the empty list;  
for (each keyword  $c$  in  $S$ ) do  
begin
```

```
     $D :=$  the list of document identifiers corresponding to  $c$ ;
```

```
    for (each document identifier  $d$  in  $D$ ) do
```

```
        if (a record  $R$  with document identifier as  $d$  is on list  $L$ ) then
```

```
             $R.\text{reference-count} = R.\text{reference-count} + 1$ ;
```

```
        else begin
```

```
            make a new record  $R$ ;
```

```
             $R.\text{document-id} = d$ ;
```

```
             $R.\text{reference-count} = 1$ ;
```

```
            add  $R$  to  $L$ ;
```

```
        end;
```

```
    end;
```

```
for (each record  $R$  in  $L$ ) do
```

```
    if ( $R.\text{reference-count} \geq k$ ) then output  $R$ ;
```

(12.24) To efficiently retrieve a document containing "leopard" using a query for "Carnivore" or "mammal", the system should use the index-time generalization. When indexing the document, it identifies the concept "leopard" and traverses the concept hierarchy upwards (e.g., to ~~Carn~~ Carnivore, then Mammal), adding the document's ID to the inverted index posting lists for all parent concepts. This makes a query for "Carnivore" a fast, direct lookup. Conversely, to allow a specific query like "leopard" to retrieve a more general document, the system should use query-time expansion. When the "leopard" query is submitted, the system traverses the hierarchy upwards and automatically rewrites the query as "leopard" OR "Carnivore" or "mammal", thus retrieving documents that match the specific term or any of its generalization.

(12.25) Based on the parameters, 1 merge pass is required after the initial sorting pass (Pass 0), as memory (10,240 blocks) is large enough to merge all 1024 initial runs $\lceil \frac{10,485,760 \text{ blocks}}{10,240 \text{ blocks/run}} \rceil$ at once.

The total cost is the sum of transfer time and seek/latency time for 2 full passes (read/write for Pass 0, read/write for Pass 1), totaling 160GB of data and $4 \times \lceil \frac{10,485,760}{10,240} \rceil$ I/O operations.

HDD Cost:

$$\rightarrow \text{Total transfer time: } (6048 / 4048) = 4096s$$

$$\rightarrow \text{Cost } (b_0 = 1)$$

$$\rightarrow \text{Seek time } (4 \times 10,485,760 \text{ sectors}) \times 0.005s/\text{seek} = 209715.2s$$

$$\rightarrow \text{Total: } 4096 + 209715.2 = 213811.2s$$

$$\rightarrow \text{Cost } (b_2 = 100)$$

$$\rightarrow \text{Seek time: } (4 \times 104,858) \times 0.0025s/\text{seek} = 209716s$$

$$\rightarrow \text{Total} = 4096 + 209716 = 619216s$$

Flash Storage (SSD) Cost:

$$\rightarrow \text{Total transfer time: } (6048 / 4048) = 409.6s$$

$$\rightarrow \text{Cost } (b_0 = 1):$$

$$\rightarrow \text{Latency time: } (4 \times 10,485,760 \text{ Hops}) \times 0.000025\mu s = 838.86s$$

$$\rightarrow \text{Total} = 409.6 + 838.86 = 1248.46s$$

$$\rightarrow \text{Cost } (b_2 = 100):$$

$$\rightarrow \text{Latency time} = (4 \times 104,858 \text{ Hops}) \times 0.000025\mu s = 8.39s$$

$$\text{Total} = 409.6 + 8.39 = 417.99s$$

(14)(a) Forcing users to choose a query strategy is undesirable because it breaks the declarative nature of SQL. SQL is designed for users to specify what data they want, not how to get it. Requiring this choice would burden users with the complex job of a query optimizer, forcing them to understand low-level details like physical storage, data structures, and join algorithms.

However, it is desirable for Database Admins (DBAs) or performance tuning developers to be aware of these costs. Query optimizers are not perfect and can create poor execution plans due to stale statistics, or query complexity. By using tools like Explain, these ~~execution plans due to stale~~ experts can analyse the optimizer's chosen strategy, identify bottlenecks, and manually intervene for example, by updating statistics, rewriting the query, or using query hints to force a more efficient plan.